



2.5. Коллекции: листы и сеты

Привет!

На связи шпаргалка урока 2.5. Коллекции: листы и сеты.

Здесь вы найдете материалы преподавателя с видеоурока, а также дополнительную информацию по теме.

Время прочтения: 22 минуты.

Коллекции

Типы коллекций

Параметризованные типы (generics)

Автоупаковка и распаковка (Autoboxing and Unboxing)

Списки (lists)

LinkedList

Как хранятся элементы

Как добавляются элементы

Как удаляются элементы

Как происходит поиск и получение элемента

Множества (sets)

Класс Collections

Печать коллекций

Итерации по коллекции

Коллекции

Ранее все задачи, подразумевающие работу с некоторым количеством объектов, мы осуществляли с помощью массивов.



Массивы — одна из основных структур данных, которая является базой для более сложных структур.

Именно такие сложные структуры мы и обсудим сегодня.

Структуры данных имеют несколько «идей», описывающих подход к хранению объектов.

Например, массив является упорядоченным хранилищем, когда в куче заранее выделяется область фиксированного размера, каждая «ячейка» этой области получает свой порядковый номер, а затем эта область заполняется объектами.

В этом случае мы можем моментально получить доступ к любой из ячеек, так как заранее знаем, что следующие n ячеек памяти принадлежат нашему массиву. Можно просто взять условную 1-ю ячейку, а затем (при необходимости обратиться к 5-й ячейке) прибавить к адресу цифру 4 и обратиться к нужной ячейке. В этом случае нам не требуется хранить ссылки на все элементы. Достаточно просто взять ссылку на первый элемент, а потом шагнуть n раз по следующим элементам.

Например, представим, что элементы массива хранятся в условных 30–35 ячейках памяти. Для обращения к последней ячейке нам требуется перейти по ссылке на самый первый элемент, в ячейку памяти № 30, а затем прибавить к этой ячейке памяти 5, что за один шаг нас переместит в ячейку памяти № 35.

Есть и другие способы хранения объектов.

Например, мы можем также положить в какую-то ссылку определенный объект, добавить этому объекту два поля (ссылка на следующий элемент и собственно сам элемент), а затем при необходимости положить в структуру новый объект класть ссылку на него в первое поле нашего стартового элемента.

Так работают связанные списки, очереди и другие структуры данных, которые мы будем изучать позже по курсу.

Сегодня ограничимся базовыми коллекциями, которые можно встретить практически в каждом проекте.

Типы коллекций



Основными типами коллекций являются:

- списки (листы),
- множества (сеты)
- словари (карты, ассоциативные массивы, мапы).

Эти типы имеют свои уникальные характеристики, достоинства и недостатки.

Чем коллекции отличаются от массивов:

- ☐ Коллекции являются более высокоуровневыми абстракциями над каким-то типом хранилища.
- ☐ В зависимости от задач они могут хранить свои данные в массиве (как String, который тоже хранит внутри массив символов) или связывать объекты друг с другом, создавая таким образом определенный порядок. Или вообще не иметь порядок, представляя собой что-то вроде «кучи» или «мешанины» из объектов.
- ☐ Коллекции обычно сами следят за своим актуальным размером (количеством заполненных ячеек или связанных объектов), сами расширяются (в случае структур на базе массивов) при определенном проценте заполнения внутреннего хранилища и имеют множество других полезных функций.
- ☐ Также коллекции можно печатать без необходимости оборачивать во что-то, как было с `Arrays.toString`, ведь коллекции являются объектами в привычном нам понимании, т. е. имеют свои методы.



Списки (lists) являются упорядоченными структурами.

Списки наиболее близки к массивам по функционалу, а самая популярная реализация списков `ArrayList` построена на базе массива.

Логика работы достаточно простая. Создаем список и добавляем туда элементы. В каком порядке добавили, в том и будут храниться. В отличие от массивов списки, даже если они основаны на базе массива, умеют расширяться. Вам не обязательно заранее создавать список определенного размера. Список можно создавать стандартного размера и, если объектов будет больше, он расширится.

Разберем две самые популярные реализации списков.



Множества (sets) являются в основном неупорядоченными структурами, особенность их в том, что все элементы множеств являются абсолютно уникальными.

Так как множества не упорядочены, мы не можем в любой момент обратиться к элементу, который был добавлен первым, вторым или последним.

По множеству можно проходить с помощью цикла, можно проверять наличие определенного элемента.



Словари (maps), они же ассоциативные массивы, представляют собой более сложные структуры данных.

Если упрощенно, то это массив из пар «ключ-значение», где ключи должны быть уникальны (как элементы множеств), а значения могут повторяться.

В качестве ключей и значений можно использовать любые типы объектов, но ключами принято делать неизменяемые типы, так как в случае изменения состояния ключа элемент может быть утерян, потому что ключ (в случае изменения) не будет соответствовать ячейке, куда была уложена наша пара ключа и значения при добавлении.

В основном ключами являются строки `String`. Значениями могут быть как строки, так и другие коллекции. Например, значением может быть список или множество.

Параметризованные типы (generics)

Возможно, вас могла заинтересовать таинственная конструкция `<E>` возле классов. Или `<K, V>` в случае с Map.



Классы, наделенные такой конструкцией, называются параметризованными типами или generics (что более популярно).

Эта конструкция позволяет нам указать тип (или типы), которые компилятор позволит нам положить в нашу коллекцию.

В случае с массивами вы писали `int[] arr`.

Коллекции типизируются через конструкцию вида `<E>`.

E — какой-то абстрактный тип.

Его задача в том, чтобы показать вам возможность использовать `generic`.

Чтобы создать коллекцию целых чисел типа `int` по примеру массива выше, нужно написать `List<Integer>`.

Заметьте, что используется не `int`, а его объектная форма `Integer`, так как примитивы указывать в качестве параметра нельзя.



Задача `generics` — соблюдать безопасность кода.

Раньше в Java их не было, а все коллекции могли содержать в себе одновременно разные типы объектов (строки и числа, например).

Ответственность за работу с коллекцией ложилась на разработчика, и временами разные разработчики могли положить в одну коллекцию разные типы объектов.

В этом случае часто возникали ошибки. Ведь один разработчик клал в коллекцию числа и ожидал получить из нее числа, а другой работал по тому же принципу, но со строками.

Вызов у числа метода `replace` или `toCharArray` выбрасывал ошибку.

Затем, когда появились `generics`, код стало писать безопаснее, ведь теперь мы не можем в список типа `Integer` положить строки. Компилятор просто даст нам по рукам.

Стоит добавить, что во время выполнения кода приложение совершенно не знает о типе коллекции, ведь `generics` существуют только на стадии компиляции. Потому `generics` на производительность не влияют.

Автоупаковка и распаковка (Autoboxing and Unboxing)

Как было упомянуто ранее, примитивы в `generics` положить нельзя.

Поэтому в generics нужно использовать классы-обертки.

У каждого примитива в Java есть своя обертка.

```
byte – Byte  
short – Short  
int – Integer  
long – Long  
float – Float  
double – Double  
char – Character  
boolean – Boolean
```

Эти обертки имеют некоторый дополнительный функционал.

Например, метод обертки `Integer parseInt` превращает строку в число типа `int`.

Метод обертки `Character isDigit` определяет, является ли символ цифрой.

И так далее.

Еще одной особенностью оберток является то, что будучи объектами, они позволяют хранить в себе `null`.

Это значит, что при создании оберток им не нужно присваивать какое-либо стандартное значение, так как оно уже задано и равно `null`.

Списки (lists)



Списки, они же листы, представляют собой умные массивы.

Они могут расширяться и сужаться, проверяться на наличие значений и сортироваться, так как в отличие от массивов списки являются классическими объектами и имеют методы.

Разберем первый по популярности и простоте список.

Он называется `ArrayList`.



Этот класс в качестве хранилища использует массив, который в случае недостатка элементов расширяется.

Но так как массивы расширять нельзя, каждый раз, когда свободные ячейки заканчиваются, создается новый массив, равный 1,5 длины старого + 1 ячейка, а затем в него копируется содержимое старого массива.

Работать с этим списком можно так же, как и с массивом.

Следующей строкой мы создаем `ArrayList`, который будем заполнять объектами типа `Integer`.

Так как мы уже знаем про полиморфизм, работать с нашим списком мы будем по его родительскому интерфейсу `List`. Это позволит в любой момент подменить `new ArrayList` на любую другую реализацию интерфейса `List` без необходимости менять остальной код, ведь список методов задает именно интерфейс.

```
List<Integer> nums = new ArrayList<>();
```

Теперь добавим в него элементы.

```
int number = 5;  
nums.add(number);
```

Метод `add` добавит нашу пятерку в конец списка.

Если необходимо добавить элемент в начало, то используется другой набор параметров того же метода `add`.

```
nums.add(0, number);
```

Первый параметр в данном случае является индексом.

Чтобы получить элемент из списка, нужно вызвать метод `get`.

```
int firstNumber = nums.get(0);
```

В этой ситуации в переменную `firstNumber` «ляжет» значение 5.

Если нам нужно заменить элемент по определенному индексу, на помощь приходит метод `set`.

```
nums.set(1, 65);
```

Код выше установит в ячейку под номером 1 значение 65, затерев то, что было в ячейке ранее.

Но учтите, что индекс должен существовать. Иначе будет выброшено исключение.

Получить количество элементов списка можно с помощью метода `size`.

```
int numsSize = nums.size();
```

В коде выше в ячейку `numsSize` будет присвоено значение, которое соответствует реальному количеству элементов в списке.

С помощью метода `contains` можно определить, имеется ли элемент в списке.

```
if (nums.contains(5)) {  
    System.out.println("В нашем списке имеется значение 5");  
}
```

Удалить элемент из списка можно методом `remove`.

```
nums.remove(5);
```

В этой ситуации удалится не элемент со значением 5, а элемент из ячейки под номером 5.

Чтобы удалить число 5 из списка чисел, нужно закастить (то есть привести один тип к другому) его к обертке.

В этой ситуации будет вызван другой метод `remove`, который удалит непосредственно объект по содержанию, а не по номеру ячейки.

```
nums.remove((Integer) 5);
```

Еще одной реализацией списка является `LinkedList`.

Отличается он тем, что в основе лежит не массив, а некий объект `Node`, который имеет поля, соответствующие следующему элементу, предыдущему элементу и значению.

В момент добавления (вызова метода `add`) данный список создает объект `Node`, укладывает в него ваш элемент, а затем встраивает в цепочку из объектов списка, заполняя внутри себя поля ссылками на прошлый элемент и следующий (если добавляем не в конец).

Создается тем же способом, что и `ArrayList`.

```
List<Integer> linkedNums = new LinkedList<>();
```

Встречается значительно реже, чем `ArrayList`.

LinkedList

Переходим к связанному списку.

Эта реализация используется редко, но в случае необходимости добавлять элементы, например, в начало дает весомый прирост производительности.

Как хранятся элементы

▼ Элементы при добавлении оборачиваются в сущность под названием `Node`, которая имеет три поля.

Первое поле хранит в себе сам объект.

Второе хранит ссылку на предыдущий `node`.

Третье хранит ссылку на следующий `node`.

```
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;

    @Contract(pure = true)
    Node(Node<E> prev, E element, Node<E> next) {
        this.item = element;
        this.next = next;
        this.prev = prev;
    }
}
```

▼ Сам список хранит только ссылки на первый и последний `node`.

```
    | Pointer to first node.
    transient Node<E> first;

    | Pointer to last node.
    transient Node<E> last;
```



Это называется двусвязный список.

Бывают еще односвязные, `node` которого хранят в себе ссылку только на следующий элемент, а сам список хранит ссылку только на первый элемент. В этом случае узлы или `node` не знают ничего о том, кто на них ссылается (идет по порядку перед ними).

Как добавляются элементы

▼ В тот момент, когда ваш код вызывает метод `add`, происходит следующее.

```

    Appends the specified element to the end of this list.
    This method is equivalent to addLast.
    Params: e – element to be appended to this list
    Returns: true (as specified by Collection.add)

    public boolean add(E e) {
        linkLast(e);
        return true;
    }

```

▼ Метод `add` вызывает метод `linkLast`, передавая ему элемент, а затем возвращает `true`.

```

    Links e as last element.

    void linkLast(E e) {
        final Node<E> l = last;
        final Node<E> newNode = new Node<>(prev: l, element: e, next: null);
        last = newNode;
        if (l == null)
            first = newNode;
        else
            l.next = newNode;
        size++;
        modCount++;
    }

```

Метод `linkLast` первым делом создает новый узел, куда помещает ваш элемент и ссылку на текущий последний элемент списка.

В этом новом узле, как мы помним из конструктора на изображении выше, имеется поле `prev`, которое хранит ссылку на предыдущий узел. Именно туда помещается ссылка на текущий последний элемент.

Затем в ссылку списка на последний элемент ложится этот новый узел. Т. е. наш новый узел становится последним элементом.

После этого предыдущему последнему узлу в поле `next` присваивается значение нашего нового созданного ранее узла.

Затем идет увеличение размера.

Добавление элемента в начало осуществляется тем же путем.

При вызове `addFirst` происходит создание нового узла, который помещается в поле `first` нашего списка. Элемент, что ранее был в поле `first` (который ранее имел в поле `prev null`) начинает ссылаться на наш новый узел в качестве предыдущего (поле `prev`).

Как удаляются элементы

▼ При вызове метода `remove` в цикле ищется узел, чей объект соответствует полю `item`. Если такой узел найден, то его «отрезают» от тех узлов, что находятся у него в полях `prev` и `next`.

```
Removes the first occurrence of the specified element from this list, if it is present. If this list does not contain the element, it is unchanged. More formally, removes the element with the lowest index i such that Objects.equals(o, get(i)) (if such an element exists). Returns true if this list contained the specified element (or equivalently, if this list changed as a result of the call).  
Params: o – element to be removed from this list, if present  
Returns: true if this list contained the specified element
```

```
public boolean remove(Object o) {  
    if (o == null) {  
        for (Node<E> x = first; x != null; x = x.next) {  
            if (x.item == null) {  
                unlink(x);  
                return true;  
            }  
        }  
    } else {  
        for (Node<E> x = first; x != null; x = x.next) {  
            if (o.equals(x.item)) {  
                unlink(x);  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Этим занимается метод `unlink`.

Его задача достаточно примитивна.

Нужно взять ссылки на предыдущий и следующий элемент из удаляемого узла, а затем прописать в предыдущий (в значение поля `next`) ссылку на следующий элемент, а следующему элементу (в значение поля `prev`) прописать предыдущий элемент.

▼ После этого сами поля удаляемого узла нулятся.

```

| Unlinks non-null node x.
E unlink(Node<E> x) {
    // assert x != null;
    final E element = x.item;
    final Node<E> next = x.next;
    final Node<E> prev = x.prev;

    if (prev == null) {
        first = next;
    } else {
        prev.next = next;
        x.prev = null;
    }

    if (next == null) {
        last = prev;
    } else {
        next.prev = prev;
        x.next = null;
    }

    x.item = null;
    size--;
    modCount++;
    return element;
}

```

Этот процесс удаляет узел из цепочки (ведь предыдущий и следующий элементы перестают на него ссылаться), а затем узел помечается в качестве мусора и удаляется.

Затем уменьшается значение `size`.

Как происходит поиск и получение элемента

- ▼ При вызове метода `get` (с передачей индекса) метод проверяет, что индекс входит в допустимые значения (индекс ≥ 0 и индекс $< \text{size}$).

```

Returns the element at the specified position in this list.
Params: index – index of the element to return
Returns: the element at the specified position in this list
Throws: IndexOutOfBoundsException –

@Contract(pure = true)
public E get(int index) {
    checkElementIndex(index: index);
    return node(index: index).item;
}

```

Далее вызывается метод `node`, возвращающий узел по индексу.

▼ В качестве результата метод `get` возвращает значение поля `item` найденного узла.

```

Returns the (non-null) Node at the specified element index.
Node<E> node(int index) {
    // assert isElementIndex(index);

    if (index < (size >> 1)) {
        Node<E> x = first;
        for (int i = 0; i < index; i++)
            x = x.next;
        return x;
    } else {
        Node<E> x = last;
        for (int i = size - 1; i > index; i--)
            x = x.prev;
        return x;
    }
}

```

Поиск по индексу осуществляется через прохождение цикла по всем элементам цепи.

В зависимости от индекса проход начинается с первого или последнего элемента.

В случае, когда нам необходимо получить первый или последний элемент, поиск не производится, происходит возврат объекта из полей списка (`first` или `last`).

Множества (sets)

Самым популярным примером множества является `HashSet`.

Это множество, которое анализирует хеш-код вашего элемента, в зависимости от него выбирает нужную ячейку в массиве и укладывает туда элемент.

Создается так же, как списки, но уже с помощью интерфейса `Set`.

```
Set<Integer> numbers = new HashSet<>();
```

Множества, в отличие от списков, не имеют методов `get` и `set`, так как индексы в них отсутствуют.

В множество можно добавить элемент с помощью метода `add`.

```
numbers.add(5);
```

Но при добавлении проверяется, имеется такой элемент внутри множества, и если имеется, элемент не добавляется в множество.

Проверить, что элемент есть в множестве, можно тем же методом `contains`.

```
if (numbers.contains(5)) {  
    System.out.println("В нашем сете имеется значение 5");  
}
```

Размер можно узнать с помощью метода `size`.

```
int numbersSize = numbers.size();
```

Удалить элемент из множества можно тем же методом `remove`, но т. к. индексы отсутствуют, кастить (то есть

привести один тип к другому) ничего не требуется.

```
numbers.remove(5);
```

Стоит запомнить, что если в вашем классе не реализованы методы `equals` и `hashCode`, `HashSet` не сможет выбрать корректную ячейку для вашего элемента и/или найти дубликаты вашего элемента.

Следовательно, главная его идея будет нарушена и вы получите обычный список, но без индексов.

Класс Collections

Класс `Collections` представляет собой аналог класса `Arrays` в отношении массивов.

В нем собраны различные статические методы, позволяющие выполнять шаблонные операции над коллекциями.

Например, в этом классе существует метод `fill`, который позволит заполнить ваш список элементами.

```
List<Integer> nums = new ArrayList<>();  
Collections.fill(nums, 1);
```

Данный код заполнит `nums` единицами по размеру списка.

Еще класс `Collections` позволяет нам получить максимальный или минимальный элемент из коллекции.

```
int max = Collections.max(nums);  
int min = Collections.min(nums);
```

Чтобы скопировать содержимое одного листа в другой, можно использовать метод `copy`.


```
List<Integer> nums = new ArrayList<>();
List<Integer> nums2 = new ArrayList<>();
nums.add(5);
nums.add(5);
nums.add(5);
Collections.copy(nums, nums2);
```

В результате оба списка будут содержать в себе три пятерки.

Печать коллекций

Так как коллекции являются полноценными объектами, в них переопределен метод `toString`.

Следовательно, мы можем их печатать путем передачи в метод `System.out.println()`.

```
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);
nums.add(3);
System.out.println(nums);
```

Будет напечатано: `[1, 2, 3]`.

Итерации по коллекции

Итерироваться по коллекциям можно с помощью циклов.

Список позволяет итерироваться с помощью циклов `for`, так как он имеет индексы, и `foreach`.

```
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);
nums.add(3);
for (int i = 0; i < nums.size(); i++) {
    System.out.println(nums.get(i));
}
```

Множество не имеет индексов, следовательно, по нему итерироваться можно только с помощью цикла `foreach`.

```
Set<String> fruits = new HashSet<>();
nums.add("apple");
nums.add("orange");
nums.add("pineapple");
for (String fruit: fruits) {
    System.out.println(fruit);
}
```

Цикл выше выведет все названия фруктов, но порядок не гарантируется.

Учтите, что модернизировать коллекции внутри `foreach` нельзя.

Следующий код приведет к выбросу исключений.

```
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);
nums.add(3);
for (int num: nums) {
    if (num % 2 == 0) {
        nums.add(num);
    }
}
```

Этот код выкинет ошибку, так как цикл `foreach` залочит (то есть заблокирует) коллекцию, по которой будет итерироваться, а добавление элемента нарушит целостность коллекции, что послужит причиной выброса исключения.

Шпаргалка урока в PDF-формате:

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/aceba800-c8b9-464a-aa23-504016d8a5c0/2.5.____.pdf